

Universidade São Judas Tadeu

Andressa Emily Rabêlo Pereira 823213904

Júlia Oliveira Rocha 823214680

Lucas Marzocca 823116813

Matheus Henrique Ferreira Guimarães 823141914

Marcos Vinicius Santos 82327399

Mylena Soares Rocha 824144075

Projeto Ticketinho — A3

Estrutura de dados e análise de algoritmos

Profº Nicolas Kassalias

São Paulo

2026

Índice

1. DESCRIÇÃO DO PROJETO	1
2. FUNDAMENTAÇÃO TEÓRICA	1
2.1 Estruturas de Dados Utilizadas	1
2.2 Padrão de Projeto Adotado	2
3. ARQUITETURA DO SISTEMA	3
3.1 Camada de Modelo (model)	3
3.2 Camada de Estruturas de Dados (estruturas)	4
3.3 Camada de Serviço (service)	4
3.4 Camada de Interface Gráfica (ui)	5
4. FLUXO DE FUNCIONAMENTO	5
4.1 Abertura de um Chamado	5
4.2 Atendimento e Resolução	6
4.3 Cancelamento e Histórico	6
4.4 Busca de Chamados	6
5. IMPLEMENTAÇÃO DO SISTEMA	6
5.1 Tecnologias Utilizadas	6
5.2 Estrutura de Pacotes	7
6. FUNCIONALIDADES DA INTERFACE	7
7. COMO EXECUTAR O SISTEMA	8
7.1 Pré-requisitos	8
7.2 Executar a partir do JAR compilado	8
7.3 Compilar e executar a partir do código-fonte	8
7.4 Executar pelo script build.sh	9
7.5 Tela inicial	9
8. EXEMPLOS DE USO	9
8.1 Abrir um novo chamado	9
8.2 Ciclo completo de atendimento	9
8.3 Busca por identificador (BST)	10
8.4 Busca por assunto (lista)	10
9. CONCLUSÃO	10
BIBLIOGRAFIA	11

1. DESCRIÇÃO DO PROJETO

O projeto Ticketinho consiste na implementação de um sistema desktop de gestão de chamados de suporte técnico, desenvolvido como trabalho prático da disciplina de Estrutura de Dados e análise de algoritmos.

O sistema permite o cadastro de usuários, equipamentos e setores, além de todo o ciclo de vida de um chamado de suporte: abertura, atendimento, resolução, cancelamento e consulta ao histórico. Cada etapa do fluxo é suportada por uma estrutura de dados específica, implementada manualmente sem o uso do Java Collections Framework.

A interface gráfica é construída inteiramente com Java Swing, seguindo um tema visual coeso com suporte a modo escuro e modo claro, que podem ser alternados em tempo de execução sem encerrar o programa.

Entre os recursos do sistema estão:

- Dashboard com indicadores de desempenho (KPIs) em tempo real;
- Painel de chamados com filtro por status e ações de atendimento;
- Painel de histórico com os chamados resolvidos e cancelados;
- Painel de cadastros para usuários, equipamentos e setores;
- Painel de busca por identificador (via BST) ou por assunto (via lista);
- Alternância de tema escuro/claro com redesenho completo da interface.

O objetivo principal é demonstrar, de forma prática, como as estruturas de dados estudadas em sala: lista ligada, fila, pilha e árvore binária de busca, participam de um sistema real de software, cada uma com uma responsabilidade distinta dentro do fluxo de operação.

2. FUNDAMENTAÇÃO TEÓRICA

Uma estrutura de dados é uma forma sistemática de organizar e armazenar dados em um computador, de modo que as operações de acesso e modificação sejam realizadas com a eficiência adequada para cada finalidade. A escolha da estrutura correta para cada problema impacta diretamente no desempenho e na clareza do código.

2.1 Estruturas de Dados Utilizadas

O sistema Ticketinho emprega quatro estruturas de dados distintas, cada uma com uma função própria no fluxo de atendimento:

Estrutura	Classe	Comportamento	Uso no Sistema
Lista Ligada	ListaLigada<T>	Acesso sequencial $O(n)$	Repositório de todos os chamados, usuários, equipamentos e setores
Fila (FIFO)	FilaChamados	Primeiro a entrar, primeiro a sair	Chamados aguardando atendimento são servidos na ordem de chegada
Pilha (LIFO)	PilhaHistorico	Último a entrar, primeiro a sair	Chamados resolvidos ou cancelados; o topo é o mais recente
Árvore BST	ArvoreBinariaChamados	Busca $O(\log n)$ no caso médio	Localização rápida de chamado por identificador numérico

A Lista Ligada genérica (ListaLigada<T>) é a estrutura mais abrangente. Ela armazena todos os chamados cadastrados e permite iteração completa, sendo usada para exibir a listagem no painel de chamados e para ordenação por prioridade.

A Fila (FilaChamados) controla a ordem de atendimento. Quando um chamado é aberto, ele é adicionado ao final da fila. Ao iniciar o atendimento, o chamado no início da fila é removido e seu status é alterado para EM_ATENDIMENTO.

A Pilha (PilhaHistorico) registra o encerramento de chamados. Sempre que um chamado é resolvido ou cancelado, ele é empilhado, de modo que o histórico exibido ao usuário mostra o chamado mais recente no topo.

A Árvore Binária de Busca (ArvoreBinariaChamados) indexa os chamados pelo identificador numérico. Ela permite localizar um chamado específico sem varrer toda a lista, com complexidade $O(\log n)$ no caso médio. A árvore não é balanceada, o que representa um ponto de atenção para volumes muito grandes de chamados com IDs sequenciais, pois pode degenerar para $O(n)$.

2.2 Padrão de Projeto Adotado

A arquitetura do sistema segue o padrão de separação em camadas. A camada de serviço (SupportService) é o único ponto de acesso às estruturas de dados, os painéis de interface não manipulam as estruturas diretamente.

O serviço é instanciado uma única vez na janela principal (JanelaPrincipal) e injetado nos painéis pelo construtor. Essa abordagem evita o uso de singletons globais e torna o código mais testável e coeso.

O sistema de temas (Tema) centraliza todas as cores, fontes e fábricas de componentes. Quando o usuário alterna entre modo escuro e claro, o tema atualiza o UIManager e redesenha toda a interface via chamada recursiva ao método repaint/revalidate, sem encerrar a aplicação.

3. ARQUITETURA DO SISTEMA

O código-fonte está organizado no pacote raiz com.suporte, dividido em subpacotes por responsabilidade. A estrutura abaixo descreve cada camada e seus componentes.

3.1 Camada de Modelo (model)

Contém as entidades do domínio da aplicação:

Classe	Descrição
Chamado	Entidade central. Representa um ticket de suporte com assunto, descrição, prioridade, status, solicitante, técnico, equipamento, setor e datas do ciclo de vida.
Usuario	Representa tanto solicitantes quanto técnicos de TI. O campo booleano tecnico diferencia os dois perfis.
Equipamento	Ativo de TI associado a um chamado. Possui tipo (Desktop, Notebook, Impressora), número de patrimônio e setor.
Setor	Departamento organizacional. Agrupa usuários e equipamentos.
Prioridade	Enumeração com quatro níveis: BAIXA, MEDIA, ALTA e CRITICA. O ordinal é usado para ordenação por gravidade.
Status	Enumeração que define o ciclo de vida do chamado: ABERTO → EM_ATENDIMENTO → RESOLVIDO ou CANCELADO → FECHADO.

Um chamado nasce sempre com status ABERTO e com a data de abertura registrada automaticamente. As datas de atendimento e fechamento são preenchidas pelo serviço no momento em que cada transição ocorre.

3.2 Camada de Estruturas de Dados (estruturas)

Contém as quatro implementações manuais, sem uso do Java Collections Framework:

Classe	Operações principais
ListaLigada<T>	adicionar(T), remover(T), get(int), tamanho(), iterator()
FilaChamados	enfileirar(Chamado), desenfileirar(), peek(), isEmpty()
PilhaHistorico	empilhar(Chamado), desempilhar(), toList()
ArvoreBinariaChamados	inserir(Chamado), buscarPorId(int)

A ListaLigada implementa a interface Iterable por meio de uma classe anônima interna, o que permite percorrê-la em laços for-each do Java. A inserção no final é $O(n)$ pois não há ponteiro para a cauda; o percurso até o último nó é realizado a cada inserção.

A FilaChamados mantém dois ponteiros: head (frente) e tail (fim). Enfileirar adiciona no tail em $O(1)$; desenfileirar remove do head em $O(1)$ e trata o esvaziamento zerando ambos os ponteiros.

A PilhaHistorico usa o ponteiro top. Empilhar e desempilhar são $O(1)$. O método toList() percorre a pilha do topo para a base e retorna uma lista de chamados na ordem mais-recente-primeiro, usada para exibir o histórico na interface.

3.3 Camada de Serviço (service)

A classe SupportService coordena todas as operações do sistema. É o único componente que conhece as quatro estruturas de dados simultaneamente.

Método	Descrição
abrirChamado(...)	Cria um Chamado, insere na ListaLigada, na FilaChamados e na ArvoreBinariaChamados.
atenderProximoChamado()	Remove o primeiro da FilaChamados e muda seu status para EM_ATENDIMENTO.
resolverChamado(id, resolucao)	Muda o status para RESOLVIDO, registra a resolução e empilha na PilhaHistorico.
cancelarChamado(id, motivo)	Muda o status para CANCELADO e empilha na PilhaHistorico.
buscarChamadoPorId(int)	Consulta a ArvoreBinariaChamados — complexidade $O(\log n)$.
buscarChamadosPorTitulo(String)	Varre a ListaLigada inteira com busca por substring — complexidade $O(n)$.

listarChamadosOrdenadosPorPrioridade()	Copia a ListaLigada para uma List Java e ordena pelo ordinal negado de Prioridade.
popularDados()	Carrega dados de demonstração na inicialização do sistema.

3.4 Camada de Interface Gráfica (ui)

Composta por sete classes Swing:

Classe	Responsabilidade
Tema	Centraliza cores, fontes e métodos fábrica para botões, campos, combos e scrollbars. Suporta modo escuro e claro.
JanelaPrincipal	Janela principal com sidebar de navegação e painel de conteúdo. Instancia o SupportService e o injeta nos painéis.
PainelDashboard	Exibe quatro KPIs (chamados abertos, em atendimento, no histórico e críticos ativos) e duas tabelas resumidas.
PainelChamados	Lista todos os chamados com filtro por status. Permite iniciar atendimento, resolver, cancelar e ver detalhes.
PainelHistorico	Exibe os chamados resolvidos e cancelados na ordem mais recente, lidos da PilhaHistorico.
PainelCadastros	CRUD de usuários, equipamentos e setores em três abas. Inserção apenas; remoção omitida por integridade referencial.
PainelBusca	Busca por identificador (BST) ou por assunto (lista). Exibe os resultados em tabela.

4. FLUXO DE FUNCIONAMENTO

4.1 Abertura de um Chamado

O usuário acessa o PainelChamados e clica em "Novo Chamado". Um formulário solicita assunto, descrição, prioridade, solicitante, equipamento e setor. Ao confirmar, o método abrirChamado() do SupportService é invocado.

Internamente, o serviço atribui um identificador único sequencial ao chamado, define o status como ABERTO e a data de abertura como o instante atual. Em seguida, insere o chamado na lista ligada (para iteração geral), na fila (para ordem de atendimento) e na árvore BST (para busca por ID).

4.2 Atendimento e Resolução

O técnico acessa o PainelChamados, seleciona um chamado com status ABERTO e clica em "Iniciar Atendimento". O serviço chama atenderProximoChamado(), que retira o chamado da fila e muda seu status para EM_ATENDIMENTO.

Após o trabalho técnico, o usuário seleciona o chamado com status EM_ATENDIMENTO e clica em "Resolver". Um diálogo solicita a descrição da resolução aplicada. O serviço registra a resolução, muda o status para RESOLVIDO, grava a data de fechamento e empilha o chamado na PilhaHistorico.

4.3 Cancelamento e Histórico

Chamados podem ser cancelados a qualquer momento. O serviço muda o status para CANCELADO e os empilha na PilhaHistorico, da mesma forma que os resolvidos.

O PainelHistorico lê a PilhaHistorico via toList() e exibe os chamados do mais recente para o mais antigo, pois o topo da pilha contém o último chamado encerrado.

4.4 Busca de Chamados

O PainelBusca oferece dois modos de pesquisa. A busca por identificador consulta diretamente a ArvoreBinariaChamados e retorna o resultado em tempo $O(\log n)$. A busca por assunto percorre toda a ListaLigada comparando substrings (case insensitive) e retorna todos os chamados cujo assunto contenha o termo digitado, com complexidade $O(n)$.

5. IMPLEMENTAÇÃO DO SISTEMA

O sistema foi desenvolvido utilizando tecnologia Java pura, sem frameworks externos:

5.1 Tecnologias Utilizadas

Tecnologia	Função
Java 21	Linguagem e plataforma de execução
Java Swing	Toolkit gráfico para construção da interface desktop
Java 2D (Graphics2D)	Renderização customizada de botões arredondados, ícones e scrollbars
java.time (LocalDateTime)	Registro e formatação das datas do ciclo de vida dos chamados
Nenhuma dependência externa	Todas as estruturas de dados e componentes são implementados do zero

5.2 Estrutura de Pacotes

```
src/
  com/suporte/
    Main.java                                # Ponto de entrada -
  SwingUtilities.invokeLater
  model/
    Chamado.java
    Usuario.java
    Equipamento.java
    Setor.java
    Prioridade.java                        # enum
    Status.java                           # enum
  estruturas/
    ListaLigada.java                       # Lista ligada genérica
    FilaChamados.java                     # Fila FIFO
    PilhaHistorico.java                   # Pilha LIFO
    ArvoreBinariaChamados.java           # BST por ID
  service/
    SupportService.java                   # Camada de negócio
  ui/
    Tema.java                             # Sistema de tema
  escuro/claro
    JanelaPrincipal.java                 # Janela principal com
  sidebar
    PainelDashboard.java
    PainelChamados.java
    PainelHistorico.java
    PainelCadastros.java
    PainelBusca.java
```

6. FUNCIONALIDADES DA INTERFACE

Funcionalidade	Descrição
Dashboard com KPIs	Quatro cartões exibem a contagem atual de chamados abertos, em atendimento, no histórico e críticos ativos. Os dados são recalculados a cada navegação.
Tabela de fila de espera	Lista os chamados aguardando atendimento com ID, assunto, prioridade e solicitante.
Tabela de chamados ativos por prioridade	Exibe os chamados em EM_ATENDIMENTO ordenados do mais crítico ao menos urgente.
Painel de chamados com filtro	Combo de filtro por status (Todos, Aberto, Em Atendimento, Resolvido, Cancelado). A tabela é recarregada ao selecionar o filtro.
Ações sobre chamados	Quatro botões: Iniciar Atendimento, Resolver, Cancelar e Ver Detalhes. Cada

	ação valida o status atual antes de executar.
Formulário de novo chamado	Diálogo modal com campos de texto, combos de prioridade, solicitante, equipamento e setor.
Histórico empilhado	Exibe chamados resolvidos e cancelados do mais recente para o mais antigo, conforme a ordem da pilha.
Cadastros em abas	Três abas (Usuários, Equipamentos, Setores) com formulários de inserção. A listagem é atualizada após cada cadastro.
Busca dupla	Campo de ID + botão para busca via BST; campo de assunto + botão para busca via lista. Resultados exibidos em tabela.
Alternância de tema	Botão na sidebar reconstrói toda a interface alternando entre modo escuro e claro, preservando o painel ativo.

7. COMO EXECUTAR O SISTEMA

7.1 Pré-requisitos

Para executar o Ticketinho é necessário:

- Java 17 ou superior instalado (recomendado Java 21);
- Variável de ambiente JAVA_HOME configurada e java no PATH do sistema operacional;
- Sem dependências externas — nenhuma biblioteca adicional é necessária.

7.2 Executar a partir do JAR compilado

Com o arquivo Ticketinho.jar disponível, execute no terminal:

```
java -jar Ticketinho.jar
```

Ou utilize o script incluso no projeto:

```
# Linux / macOS
bash run.sh
```

```
# Windows (PowerShell)
.\run.sh
```

7.3 Compilar e executar a partir do código-fonte

Caso o JAR não esteja disponível, compile manualmente com os comandos abaixo:

```
# 1. Criar o diretório de saída
mkdir -p out
```

```
# 2. Listar todos os arquivos .java
find src -name "*.java" > sources.txt

# 3. Compilar
javac --release 17 -d out @sources.txt

# 4. Criar o MANIFEST.MF
echo "Main-Class: com.suporte.Main" > MANIFEST.MF

# 5. Empacotar
jar cfm Ticketinho.jar MANIFEST.MF -C out .

# 6. Executar
java -jar Ticketinho.jar
```

7.4 Executar pelo script build.sh

O projeto inclui um script build.sh que automatiza a compilação e execução:

```
bash build.sh
```

O script localiza o JDK no JAVA_HOME, compila todos os fontes em Java 21, gera o Ticketinho.jar e inicia a aplicação automaticamente.

7.5 Tela inicial

Ao iniciar, o sistema carrega automaticamente dados de demonstração: cinco setores, três equipamentos, cinco usuários (dois técnicos e três solicitantes) e seis chamados em diferentes status. O Dashboard é exibido como tela inicial.

8. EXEMPLOS DE USO

8.1 Abrir um novo chamado

No PainelChamados, clicar em "Novo Chamado" e preencher os campos:

```
Assunto      : Impressora não imprime
Descrição   : A impressora HP LaserJet do financeiro não responde
aos comandos de impressão.
Prioridade  : Alta
Solicitante : João Pereira
Equipamento: Impressora HP LaserJet (PAT-003)
Setor       : Financeiro
```

Ao confirmar, o sistema registra o chamado com ID gerado automaticamente, status ABERTO, e o adiciona à fila de espera.

8.2 Ciclo completo de atendimento

Sequência de ações no painel de chamados:

1. Selecionar chamado com status ABERTO e clicar em "Iniciar Atendimento", status muda para EM_ATENDIMENTO;
2. Após o trabalho técnico, selecionar o chamado e clicar em "Resolver";
3. Informar a resolução aplicada no campo de texto do diálogo;
4. Confirmar, status muda para RESOLVIDO e o chamado aparece no PainelHistorico.

8.3 Busca por identificador (BST)

No PainelBusca, digitar o ID e clicar em "Buscar por ID":

ID: 3

Resultado:

ID	Assunto	Status	Prioridade
Solicitante			
3	Lentidão no computador	EM_ATENDIMENTO	Alta
Ana Ferreira			

A consulta percorre a ArvoreBinariaChamados comparando o ID digitado com a chave de cada nó, descendo à esquerda se menor e à direita se maior, com complexidade $O(\log n)$ no caso médio.

8.4 Busca por assunto (lista)

No PainelBusca, digitar um termo e clicar em "Buscar por Assunto":

Assunto: impressora

Resultado:

ID	Assunto	Status	Prioridade
Solicitante			
4	Impressora sem papel	ABERTO	Baixa
Pereira			
7	Impressora não imprime	ABERTO	Alta
Pereira			

9. CONCLUSÃO

O desenvolvimento do projeto Ticketinho permitiu aplicar na prática os conceitos estudados na disciplina de Estrutura de Dados e Programação Orientada a Objetos, especialmente a construção e o uso de estruturas de dados implementadas do zero para resolver problemas reais de um sistema de software.

A partir da definição de um domínio concreto (gestão de chamados de suporte técnico) foi possível motivar a escolha de cada estrutura de dados com base no comportamento necessário: a fila para respeitar a ordem de chegada dos

chamados, a pilha para registrar o histórico do mais recente ao mais antigo, a árvore binária para acelerar a busca por identificador e a lista ligada para servir de repositório geral com suporte à iteração completa.

A implementação evidenciou a diferença de complexidade entre as estruturas: enquanto a busca por ID na BST opera em $O(\log n)$ no caso médio, a busca por assunto na lista ligada exige $O(n)$. Essa distinção, facilmente percebida na prática, demonstra por que a escolha adequada da estrutura de dados é fundamental para o desempenho de um sistema.

A camada de interface construída com Java Swing demonstrou como separar responsabilidades em uma aplicação desktop: a camada de serviço concentra toda a lógica de negócio e gerencia as estruturas de dados, os painéis de interface apenas exibem dados e delegam ações ao serviço. O sistema de tema com suporte a modo escuro e modo claro exemplificou como centralizar decisões visuais em uma única classe facilita a manutenção e a consistência visual da aplicação.

O projeto demonstrou que as estruturas de dados estudadas em sala não são exercícios abstratos, elas compõem a espinha dorsal de sistemas reais. A experiência consolidou o entendimento sobre como lista ligada, fila, pilha e árvore binária de busca podem coexistir em uma mesma aplicação, cada uma servindo a uma finalidade específica dentro de um fluxo de trabalho coeso.

BIBLIOGRAFIA

CORMEN, T. H. et al. Introduction to Algorithms. 3. ed. MIT Press, 2009. Cap. 10 (Elementary Data Structures), Cap. 12 (Binary Search Trees).

SEDGEWICK, R.; WAYNE, K. Algorithms. 4. ed. Addison-Wesley, 2011. Cap. 1.3 (Bags, Queues, and Stacks), Cap. 3.2 (Binary Search Trees).

GOODRICH, M. T.; TAMASSIA, R.; GOLDWASSER, M. H. Data Structures and Algorithms in Java. 6. ed. Wiley, 2014. Cap. 7 (List and Iterator ADTs), Cap. 8 (Trees), Cap. 11 (Search Trees).

DEITEL, H.; DEITEL, P. Java: Como Programar. 10. ed. Pearson, 2017. Cap. 17 (Estruturas de Dados Genéricas — listas, filas, pilhas, árvores).

GeeksforGeeks. Implementing a Binary Tree in Java (2024).
<https://www.geeksforgeeks.org/java/implementing-a-binary-tree-in-java/>

Runestone Academy. 6.14 Search Tree Implementation — javads.
https://runestone.academy/ns/books/published/javads/trees_search-tree-implementation.html

Educative.io. Deletion in Binary Search Tree (BST) in Java.
<https://www.educative.io/answers/deletion-in-binary-search-tree-bst-in-java>

ManageEngine ServiceDesk Plus. IT Ticketing System — How It Works with ITIL and ITSM (2025). <https://www.manageengine.com/products/service-desk/help-desk-software/it-helpdesk-ticketing-system.html>

InvGate Blog. How to Use Help Desk Priority Levels (2025).
<https://blog.invgate.com/help-desk-priority-levels>

Ivanti. 20 IT Ticket Handling Best Practices (2025). <https://www.ivanti.com/blog/it-ticket-handling-best-practices>

Oracle. The Java™ Tutorials — Creating a GUI with Swing.
<https://docs.oracle.com/javase/tutorial/uiswing/>

CMU. Advanced BST Operations — Traversals, Delete, AVL Trees.
https://www.cs.cmu.edu/~clo/www/CMU/DataStructures/Lessons/lesson4_3.htm